FINAL-YEAR GROUP PROJECT · INDEPENDENT REPOSITORY REVIEW

Project review and improvement recommendations

Missing parts, confirmed mistakes, research gaps and a practical completion plan

The project has a substantial implementation. The most valuable next work is reliable assessment, secure interview handling and measured evidence of quality.

Review detail	Scope
Project	CSS3223_Project_AI-CV-Analysis-System / Vynterva AI
Review date	4 November 2026 · Asia/Colombo
Repository snapshot	Working tree at commit da49463; source was clean when review began
Method	Read-only source/configuration review, existing test runs, isolated offline probes
Deliverable	Review document only. No application source, configuration or tests changed.

Prepared for the project group. This is a technical and academic readiness review, not an awarded grade, penetration-test certification or legal opinion.

EXECUTIVE ASSESSMENT

What your group should do next

You already have more than a basic CV analyzer: a multi-service application with candidate preparation, recruiter-controlled assessments, applications, decisions and administration. The main weakness is evidence and reliability around the core AI assessment workflow, rather than a lack of screens.

First fix session authorization and durable answer handling. Then prove that scores are valid, recoverable and reasonably aligned with human judgment. Keep the remaining scope focused.

Order	Recommended outcome
1 · Immediate	Close cross-account interview access before using other people's CVs or answers (F01).
2 · Before final demo	Make answer/retry behavior reliable, enforce exam timing on the server, validate grading output, and prove a fresh RAG startup (F02–F08).
3 · Before submission	Run a small controlled AI evaluation and usability study; align the dissertation, diagrams and setup guide with the final implementation.
4 · Before public use	Complete privacy information, retention handling, operational controls, dependency reproducibility and backup restoration.

How to interpret this report

Confirmed means the behavior is visible in current source or reproduced by an offline check. **Inferred impact** means the likely consequence has not been exercised on a running stack. **Missing** means no supporting artifact was found in the reviewed repository; it may exist elsewhere in your group's work.

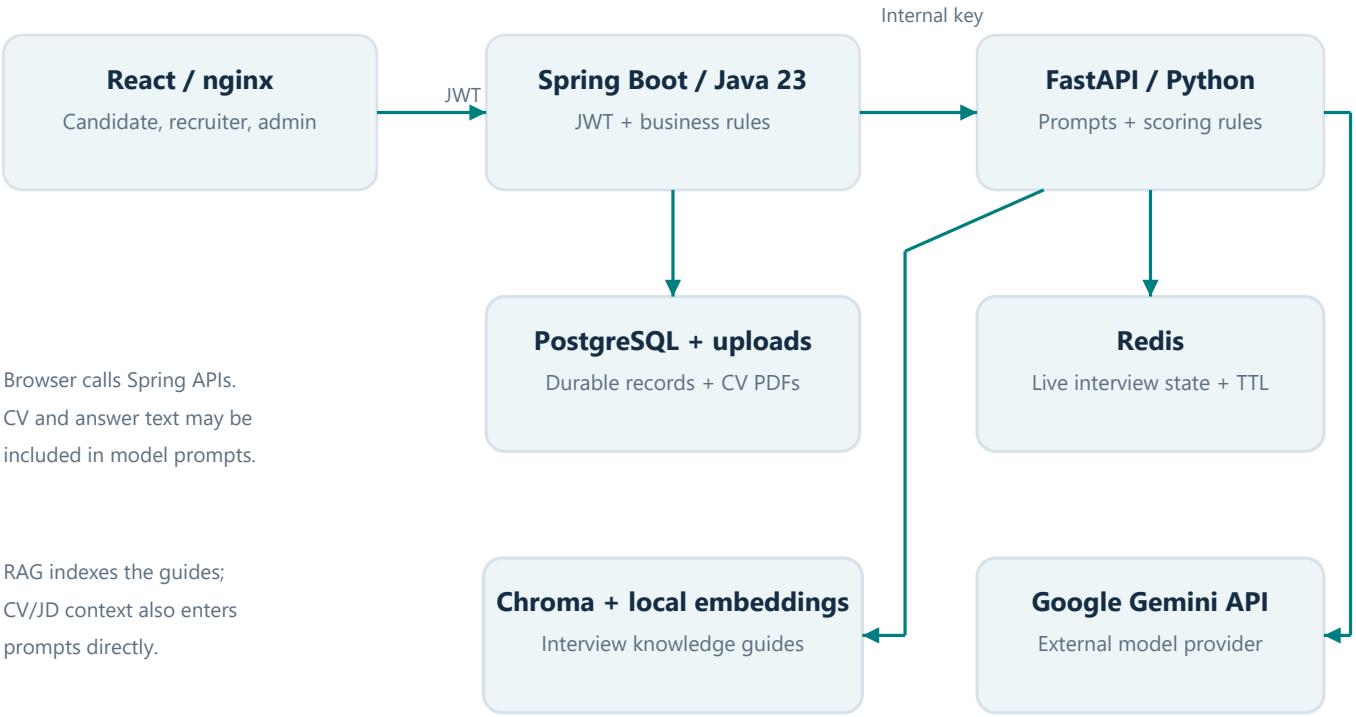
Priorities: P0 = immediate access-control repair; P1 = core correctness and demonstration reliability; P2 = completion, quality and deployment readiness. Severity is an engineering judgment, separate from your university's marking rubric.

Navigation

Pages 3–6: system, strengths, verification and issue register.
Pages 7–14: sixteen detailed findings with evidence and acceptance checks.
Pages 15–18: missing academic work, AI experiments and test plan.
Pages 19–24: operations, UX, group roadmap, viva, submission checklist and references.

ARCHITECTURE

The system that is actually implemented



Candidate preparation: PDF upload → text extraction/OCR → structured CV analysis → skill gap → adaptive practice → scorecard → learning roadmap.

Recruitment: approved recruiter → job requirements and approved fixed questions → public/private applications → manual shortlisting → comparable fixed interview → recruiter decision → candidate notification.

Important distinction: practice interviews adapt to CV/context. Job interviews use the recruiter's approved fixed questions, helping comparison within a job. Those two flows should have separate claims and evaluation criteria.

RAG: Chroma stores chunks from ai-service/documents. Embeddings are local, using all-MiniLM-L6-v2. The implementation passes CV/job context into prompts directly; the older architecture description of indexing each CV/JD with Gemini embeddings is inaccurate.

Sources: docker-compose.yml; frontend/src/services/api.js; backend/pom.xml; ai-service/app/rag/embeddings.py; ai-service/app/interview/session_manager.py; backend service implementations.

WHAT IS ALREADY PRESENT

Keep these strengths in your final submission

Area	Observed implementation
Domain workflow	Candidate/recruiter/admin roles; recruiter approval; jobs, CV-linked applications, shortlist/reject, fixed assessments, hiring decisions and notifications.
Security foundation	BCrypt passwords, JWT validation, method security, internal AI key, account/AI rate limits, and many owner-scoped domain services.
Persistence	PostgreSQL records, Flyway V1–V13, stored interview turns/results, CV analysis snapshots, and Redis live sessions with idle expiry.
AI design	Central model configuration/fallback; deterministic skill arithmetic and score aggregation; answer fencing; guide retrieval; reference excerpts; CV bullet tuner and roadmap.
Candidate experience	Typed/voice input, spoken questions, practice/exam modes, browser timer auto-submit, error recovery, history, print views, themes and accessibility primitives.
Engineering	Three CI workflow files, Dockerfiles, frontend/backend/AI tests and a running-stack security-check script.

Do not repeat outdated allegations as current bugs

• Redis storage and TTL now exist. AI-process restarts are different from Redis restarts; fixed interview restoration also exists.

• The normal upload flow reads stored analysis instead of always paying for a second extraction call. CV quality review is a separate feature/call.

• Client file-size checks, password rules and exam auto-submission exist. Recruiter and candidate report views are implemented.

• CI exists. The plan’s “no CI workflow yet” checkbox and older test counts are stale.

Presence in source does not prove full runtime reliability or scoring accuracy. The issues that remain are described on the following pages.

VERIFICATION RECORD

What was checked, and what passed

Check	Result and interpretation
Frontend existing suite	**207 tests passed across 20 suites.** Ran npm.cmd test -- --watchAll=false --runInBand with CI=true. React act deprecation warnings and expected error-path logs appeared; they were not test failures.
Backend existing suite	**290 tests passed, zero failures/errors/skips** using JDK 23 and offline Maven. The first Java 25 run had Byte Buddy/Mockito compatibility errors; rerunning with the documented JDK resolved them. This was not evidence of 64 product defects.
AI local subset	**31 tests passed** for test_session_ttl and test_skill_matcher. They run with the existing workspace venv without provider requests.
AI broader attempt	Incomplete: a run excluding test_rag reported 17 errors in 49 loaded test entries, due to missing LangChain, FastAPI, dotenv, redis and related packages. Some entries were import failures. This is an environment limitation, not a valid whole-suite pass/fail judgment.
Offline source probes	Actual evaluator functions accepted {} as evaluated=true / score=0. Legacy quality rules labeled a short precise answer weak and 40 irrelevant words strong. Empty-index opening raised the documented missing-Chroma exception.
Repository inventory	26 backend test files; 20 frontend test files; 125 statically declared AI test methods. Method counts are inventory, not a passed-test total.

Limits of this review

No full-stack browser session, fresh Docker deployment, live Gemini call, real PostgreSQL migration test, OCR fixture run, load test or exploit attempt was performed. No real candidate records were modified. Live service behavior and visual usability remain to be demonstrated.

The isolated Python probes execute selected source functions with stubbed dependencies. They verify those branches, not the complete FastAPI application. Existing frontend tests use a simulated DOM; backend tests do not replace PostgreSQL integration tests.

Application source, existing tests and configuration were left unchanged. Test runs wrote ordinary ignored build/cache artifacts. This PDF is the only requested repository deliverable.

SIXTEEN FINDINGS

Prioritized issue register

ID	Priority / severity	Finding	Suggested owner
F01	P0 / Critical	Session ownership is checked after the AI action	Backend
F02	P1 / High	A successful response can hide a failed database write	Backend + AI
F03	P1 / High	Retries and concurrent requests can advance the wrong turn	Backend + AI
F04	P1 / High	Exam time is enforced only by the browser	Backend + Frontend
F05	P1 / High	Malformed grading JSON can become a successful zero	AI
F06	P1 / High	A temporary grading failure becomes a final cached result	AI + Backend
F07	P1 / High	A fresh installation has no automatic RAG index bootstrap	AI + Deployment
F08	P1 / High	Ordinary AI queries can request a shared index rebuild	Backend + AI
F09	P2 / Medium	A fallback CV analysis is displayed but not stored	Backend + Frontend
F10	P2 / Medium	Answer quality uses inconsistent rules for adaptation	AI
F11	P2 / Medium	The RAG API ignores the requested top_k	AI
F12	P2 / Medium	Document and text workloads lack practical upper bounds	Backend + AI
F13	P2 / Medium	Personal-data handling is only partly explained and managed	Group lead + Backend + Frontend
F14	P2 / Medium	AI dependency versions and experiments are not reproducible	AI + Deployment
F15	P2 / Medium	Architecture and plan documents contradict the implementation	Documentation lead + all members
F16	P2 / Medium	Local deployment defaults need an explicit public-use profile	Deployment + Backend

P0: 1 finding · P1: 7 findings · P2: 8 findings. Effort in the detail pages is relative: S = localized work, M = cross-layer work, L = substantial coordination. These are planning hints, not promised delivery dates.

Fix F01 first. Coordinate F02 and F03, because changing response handling without idempotency can make retries unsafe. F05 and F06 belong in the same grading-state design.

EVIDENCE AND RECOMMENDED CORRECTION

Findings F01–F02

F01 · Session ownership is checked after the AI action

P0 / Critical · Backend · Relative effort M

Observed. The answer and finish routes forward the supplied session_id to FastAPI before any ownership check. The recorder checks the owner only afterwards and returns without stopping the response. FastAPI identifies the live session by ID and has no end-user identity.

Why it matters. A signed-in caller who obtains another active session ID can submit to it or finish it. A finish response can include the other candidate's answers and feedback. UUID secrecy and a shared internal key do not establish ownership.

Recommendation. Load the session through the authenticated owner before every session read or mutation, and reject unauthorized access before calling Python. Apply the same principle to question/model-answer routes that accept a session ID.

Acceptance check. With two test accounts, every cross-account session request is refused, makes zero AI calls, returns no transcript, and leaves both Redis and PostgreSQL unchanged.

Evidence status: Confirmed by static control-flow review

```
backend/src/main/java/com/aiinterview/controller/AiProxyController.java:159–185, 268–315
backend/src/main/java/com/aiinterview/service/impl/InterviewPersistenceServiceImpl.java:330–358
ai-service/app/main.py:1434–1444, 2080–2158
```

F02 · A successful response can hide a failed database write

P1 / High · Backend + AI · Relative effort M–L

Observed. forwardAndRecord() catches persistence exceptions, logs them, and still returns the successful AI response. Python can therefore advance or complete an interview while the durable database record remains unchanged.

Why it matters. History, recruiter results, application status and recovery can disagree with what the candidate saw. A failed start write can leave an AI session without an owner record; a failed answer write can disappear after Redis loss.

Recommendation. Define when an answer is durably accepted. Record a command/turn with a stable ID, process it once, and reconcile incomplete AI results. Return a recoverable pending state when saving is incomplete. Use idempotency together with this change so a retry cannot advance twice.

Acceptance check. Inject a database failure after AI success. The UI must not report durable completion; recovery must eventually produce exactly one matching turn and one scorecard.

Evidence status: Confirmed behavior; failure injection not run

```
backend/src/main/java/com/aiinterview/controller/AiProxyController.java:296–315
backend/src/main/java/com/aiinterview/service/impl/InterviewPersistenceServiceImpl.java:138–246
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F03–F04

F03 · Retries and concurrent requests can advance the wrong turn

P1 / High · Backend + AI · Relative effort M–L

Observed. Answer requests contain session_id and answer but no expected question/turn ID or idempotency key. Fixed interviews append the answer and advance by the current answer count. Session storage uses read-modify-write without a per-session lock or version check.

Why it matters. Replaying a request after a lost response can attach the same answer to the next question. Concurrent requests can overwrite each other. Concurrent starts can both pass eligibility before the database's one-attempt constraint rejects one recording; F02 still returns its AI result.

Recommendation. Require a stable request ID and expected turn/version; save the first outcome and replay it on duplicate requests. Serialize mutations for one session. Reserve the job attempt atomically before creating the AI session.

Acceptance check. Duplicate, reordered and simultaneous submissions produce one accepted answer for the intended question. Two simultaneous starts produce one authorized attempt and no usable orphan session.

Evidence status: Confirmed missing safeguards; race impact inferred

```
ai-service/app/interview/schemas.py:124–134
ai-service/app/main.py:1256–1295
ai-service/app/interview/session_manager.py:152–165, 301–311
backend/src/main/resources/db/migration/V11__job_applications.sql:177–179
```

F04 · Exam time is enforced only by the browser

P1 / High · Backend + Frontend · Relative effort M

Observed. InterviewRoom starts a local countdown from TOTAL_TIME when a question is shown and resets it after remount/resume. The reviewed answer request, Python session model and backend turn handler contain no authoritative answer deadline.

Why it matters. Reloading or altering browser state can extend the time available. A suspended/background tab can also delay a countdown based on timer ticks. Auto-submission exists, but it does not enforce a server-side exam rule.

Recommendation. Store question-issued time, deadline and an agreed network grace period on the server. Return the deadline to the UI, calculate remaining time from it, and enforce it for direct API calls as well as browser submissions.

Acceptance check. Reload, tab suspension, clock changes and direct API submissions cannot extend the deadline. Late answers follow one documented policy; accommodations can be configured explicitly.

Evidence status: Confirmed by static review; browser bypass not exercised

```
frontend/src/pages/InterviewRoom.jsx:552–583, 1346–1352
ai-service/app/interview/schemas.py:124–134
ai-service/app/interview/session_manager.py:17–135
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F05–F06

F05 · Malformed grading JSON can become a successful zero

P1 / High · AI · Relative effort S–M

Observed. The evaluator accepts any JSON object, fills absent scores with zero, and then sets evaluated=True. An isolated execution of the actual scoring functions, with a fake model returning {}, produced evaluated=true and overall_score=0.

Why it matters. A model-format failure can be recorded as poor candidate performance. Valid arithmetic over missing scores does not make the assessment valid. Recruiter averages may then include a zero that no candidate earned.

Recommendation. Validate the complete model output before score normalization: required category keys, finite numeric ranges, one entry for each evaluated question, unique question numbers, and usable feedback. Treat a failed contract as unscored and retry or queue review.

Acceptance check. {}, missing categories, duplicate/missing questions, nonnumeric values and non-finite scores all produce an explicit unscored outcome. A valid earned zero remains distinguishable from a processing failure.

Evidence status: Reproduced using isolated source functions and a fake model

```
ai-service/app/interview/evaluator.py:463–544, 672–684
backend/src/main/java/com/aiinterview/service/impl/InterviewPersistenceServiceImpl.java:224–229
```

F06 · A temporary grading failure becomes a final cached result

P1 / High · AI + Backend · Relative effort M

Observed. A failed evaluation returns a nonempty dictionary with evaluated=False. Finish saves it on a completed session. A repeated finish returns any cached evaluation, including that failure. The database recorder also stops updating an already completed interview.

Why it matters. An interview may remain unscored after a transient provider outage even when the service recovers. The current null-score handling is helpful, but there is no reviewed regrade workflow to finish the assessment later.

Recommendation. Separate interview completion from grading status: pending, succeeded, failed and reviewed. Allow bounded retry or an authorized regrade of failed evaluations, retaining the original answers and an audit trail. Cache successful results without repeatedly charging the model.

Acceptance check. Simulate a failed first grading and a successful retry. Exactly one final valid result reaches candidate history and recruiter views; previously successful results do not change on ordinary repeat requests.

Evidence status: Confirmed by static review

```
ai-service/app/main.py:2098–2143
ai-service/app/interview/evaluator.py:559–595, 694–702
backend/src/main/java/com/aiinterview/service/impl/InterviewPersistenceServiceImpl.java:212–229
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F07–F08

F07 · A fresh installation has no automatic RAG index bootstrap

P1 / High · AI + Deployment · Relative effort M

Observed. The image caches the embedding model and creates an empty chroma_db directory. Interview retrieval opens the vector store without documents; that function raises if no index exists. No application startup hook or Compose indexing step was found. The health endpoint still reports healthy.

Why it matters. The advertised fresh setup can reach a healthy state while practice retrieval fails. Some other paths catch retrieval errors and continue without grounding, which can hide the missing knowledge base.

Recommendation. Add an explicit, authorized bootstrap/index preparation stage and a separate readiness check for a usable nonempty corpus. Record the corpus hash and embedding configuration. Document how to rebuild without disrupting active queries.

Acceptance check. On a fresh disposable volume, following the README alone makes the first practice interview retrieve real guide passages. Readiness stays false until indexing finishes. An empty-index source-function probe reproduced the current exception.

Evidence status: Missing-index branch reproduced in isolation; full Docker start not run

```
ai-service/Dockerfile:31–59
ai-service/app/rag/vector_store.py:69–79, 111–171
ai-service/app/interview/question_generator.py:1086–1100
ai-service/app/main.py:402–415
```

F08 · Ordinary AI queries can request a shared index rebuild

P1 / High · Backend + AI · Relative effort S–M

Observed. The candidate-accessible RAG query proxy forwards arbitrary JSON. Its request schema includes force_rebuild, which reaches the vector store. A true value deletes the existing index directory before rebuilding it. The proxy requires login but has no administrator-only guard.

Why it matters. An ordinary account can trigger expensive maintenance of a shared resource, potentially interrupting retrieval for other users. A failed or concurrent rebuild can leave the index unavailable.

Recommendation. Remove maintenance flags from user query contracts. Put rebuild operations behind an administrator or deployment-only path, rate-limit them separately, build a new version separately, and switch only when it is validated.

Acceptance check. A candidate's query cannot delete or rebuild the index. Authorized rebuilding preserves the previous usable version until the replacement passes validation.

Evidence status: Confirmed code path; destructive operation was not executed

```
backend/src/main/java/com/aiinterview/controller/AiProxyController.java:234–239
ai-service/app/main.py:314–322, 445–448
ai-service/app/rag/vector_store.py:86–105
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F09–F10

F09 · A fallback CV analysis is displayed but not stored

P2 / Medium · Backend + Frontend · Relative effort M

Observed. If upload-time analysis is unavailable, CVUpload calls the generic AI analyze proxy and stores the response in browser sessionStorage. That proxy only forwards the request. Applications later take their evidence from the database's ResumeAnalysis record.

Why it matters. The candidate can see a successful analysis while a later application has no matching stored analysis snapshot or receives an empty-skill match. This is a failure-path gap; the normal upload path already reuses saved analysis and avoids the old unconditional duplicate call.

Recommendation. Provide an owner-checked analyze/retry operation for the stored resume and persist its result before reporting success. Return processing/analysis status explicitly. Derive future application evidence from that same saved version.

Acceptance check. Fail the upload-time model call, recover, then analyze and apply. The candidate result, stored analysis and recruiter snapshot must agree after browser storage is cleared.

Evidence status: Confirmed failure-path inconsistency by static review

```
frontend/src/pages/CVUpload.jsx:229–240, 138–142
backend/src/main/java/com/aiinterview/controller/AiProxyController.java:61–66
backend/src/main/java/com/aiinterview/service/impl/JobApplicationServiceImpl.java:167–180
```

F10 · Answer quality uses inconsistent rules for adaptation

P2 / Medium · AI · Relative effort M

Observed. Difficulty uses assess_answer() with correctness context, but the weak-answer streak and retrieval instructions still use the older length heuristic. Both paths force answers under eight words into weak. A short precise example was classified weak, while forty irrelevant words were classified strong by the legacy function.

Why it matters. Brief correct answers can force easier questions or topic changes. The same answer can have one quality for difficulty and another for the weak streak. The system should not claim that all adaptive behavior is based on correctness.

Recommendation. Use one recorded quality assessment for difficulty, streak, topic selection and explanations. Limit rule-based weak labels to genuine non-answers; permit concise correct responses. Mark any fallback explicitly.

Acceptance check. Test short correct, long wrong, off-topic and non-answer cases. All adaptive decisions use the same quality result and record the source of that decision.

Evidence status: Legacy heuristic reproduced in isolation; whole adaptive turn not run

```
ai-service/app/main.py:1546–1579
ai-service/app/interview/question_generator.py:897–917, 948–954, 1206–1208, 1997–2003
ai-service/app/interview/answer_quality.py:188–205
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F11–F12

F11 · The RAG API ignores the requested top_k

P2 / Medium · AI · Relative effort S

Observed. RAGQueryRequest validates top_k from 1 to 10, but query_rag_pipeline() calls run_rag_query() with query and force_rebuild only. The requested retrieval count is not forwarded, so the pipeline default is used.

Why it matters. Clients and evaluation experiments cannot reliably choose the retrieval depth promised by the contract. A k=1 versus k=5 comparison could accidentally measure the same configuration twice.

Recommendation. Pass the validated retrieval count through the route and return the effective value in experiment metadata. Ensure the passages reported with an answer are the same passages used to generate it; the general RAG pipeline currently retrieves twice.

Acceptance check. Requests for top_k=1 and top_k=5 reach the retriever with those exact values, subject only to available corpus size. Returned source evidence matches the context provided to the model.

Evidence status: Confirmed by static review

```
ai-service/app/main.py:306–312, 445–448
ai-service/app/rag/pipeline.py:303–307, 389 onward
```

F12 · Document and text workloads lack practical upper bounds

P2 / Medium · Backend + AI · Relative effort M

Observed. Uploads have a 10 MB limit, but OCR loops over every page and renders at 300 DPI without a reviewed page-count, image-size or processing-time budget. Several AI request text fields have minimum lengths without maximum lengths. PDF extraction and AI calls are synchronous.

Why it matters. A small compressed PDF can still require substantial rendering memory or CPU. Long JSON inputs and slow model calls can hold request threads and database transactions. Per-account request limits do not bound the work inside each request.

Recommendation. Set tested page, dimension, extracted-text, token and duration limits. Move lengthy processing to a bounded worker/job flow where needed. Avoid holding database transactions while awaiting the model; expose pending, failed and retryable states.

Acceptance check. Large-page, many-page, encrypted, corrupted and oversized-text fixtures fail clearly within stated time and memory budgets. A slow upload does not stall unrelated candidate requests.

Evidence status: Confirmed missing local bounds; resource exhaustion not tested

```
backend/src/main/java/com/aiinterview/service/PdfTextExtractionService.java:43–59, 69–106
backend/src/main/java/com/aiinterview/service/impl/ResumeAnalysisServiceImpl.java:45–75
ai-service/app/interview/schemas.py:55–93, 124–134
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F13–F14

F13 · Personal-data handling is only partly explained and managed

P2 / Medium · Group lead + Backend + Frontend · Relative effort M

Observed. Job applications already store consent to recruiter sharing, including its version and time. However, no Privacy/Terms routes, complete retention schedule, account export/deletion flow, or upload-stage explanation of external AI processing was found. Submitted CV deletion is deliberately refused.

Why it matters. Candidates cannot see the full lifecycle of their CV text, answers, stored evidence and provider requests. Existing recruiter-sharing consent does not explain every processing stage or how retained application evidence is handled.

Recommendation. Document the data flow before using real participant CVs. Explain provider processing, recipients, retention, withdrawal/request handling and limits on deletion. Minimize personal details in prompts and logs. Use synthetic or specifically consented research data.

Acceptance check. A participant can understand who receives each data category, how long it is retained and how to make a data request. The implemented behavior and participant information sheet agree. Obtain institution-specific ethics guidance.

Evidence status: Repository gap; no legal-compliance conclusion is made

```
frontend/src/components/Footer.jsx:44–48
frontend/src/pages/JobApplication.jsx:395–399
backend/src/main/java/com/aiinterview/entity/JobApplication.java:140–151
backend/src/main/java/com/aiinterview/service/impl/ResumeServiceImpl.java:191–194
```

F14 · AI dependency versions and experiments are not reproducible

P2 / Medium · AI + Deployment · Relative effort S–M

Observed. requirements.txt uses broad lower bounds for FastAPI, LangChain, Chroma, transformers and model tooling, with no reviewed lock file. Docker separately installs an unpinned torch. The current workspace venv is missing multiple required packages. Model fallback is centralized, but experiment results need the model that actually answered.

Why it matters. A fresh installation may resolve a different dependency combination or change behavior without source changes. Results cannot be fairly compared if different fallback models or corpus versions are mixed without being recorded.

Recommendation. Produce a tested dependency lock/constraints set, record the Python/JDK/Node toolchain and image identifiers, and save effective model, prompt version, corpus hash and settings with evaluation runs. Reproduce on a clean environment before submission.

Acceptance check. A clean runner installs the same tested versions and executes the full AI suite without real credentials. Every research result identifies the actual model and retrieval configuration used.

Evidence status: Versioning gap confirmed; full AI tests locally blocked by missing dependencies

```
ai-service/requirements.txt:1–30
ai-service/Dockerfile:31–36
.github/workflows/ai-service-ci.yml
ai-service/app/llm_config.py
```

EVIDENCE AND RECOMMENDED CORRECTION

Findings F15–F16

F15 · Architecture and plan documents contradict the implementation

P2 / Medium · Documentation lead + all members · Relative effort S–M

Observed. ARCHITECTURE.md describes Gemini embeddings, Chroma/FAISS, and CV/JD indexing. Current code uses local HuggingFace embeddings and Chroma over guide documents. Other docs say Java 17; the actual baseline is 23. PROJECT_PLAN says CI is absent and lists old test totals and migrations V1–V8; three CI workflows and migrations through V13 exist.

Why it matters. An examiner following the documents can misunderstand the contribution or fail to reproduce the project. Describing planned work as implemented weakens the viva even when the actual implementation is substantial.

Recommendation. Make one current architecture/SRS/API set, mark planned or deliberately excluded features, use relative document links, and reconcile names, toolchain, data flow, migrations and test evidence. Treat old investigation reports as historical snapshots.

Acceptance check. A member unfamiliar with the implementation can explain and start it using the documentation alone. Every claimed feature and measured result has current evidence.

Evidence status: Confirmed documentation inconsistencies

```
ARCHITECTURE.md:44–70
PROJECT_PLAN.md:21–22, 42–46
docs/README.md:9–13
.github/workflows/
backend/src/main/resources/db/migration/
```

F16 · Local deployment defaults need an explicit public-use profile

P2 / Medium · Deployment + Backend · Relative effort M

Observed. Compose publishes PostgreSQL on a host port and defaults to the example database password; Spring connects using the postgres account. The shipped nginx profile serves HTTP. Health endpoints mainly report process/configuration status. No backup/restore drill, load-test evidence or production monitoring setup was found.

Why it matters. These defaults suit a controlled local demonstration but do not establish readiness for internet-facing candidate data. Volumes preserve files across container replacement, not across host loss or operator error.

Recommendation. Use a distinct deployment profile with HTTPS termination, private database access, generated credentials, a limited database role and dependency-aware readiness. Back up PostgreSQL and uploaded CVs consistently; prove restore on a separate test instance. Add request/latency/failure metrics.

Acceptance check. The intended deployment exposes only required public endpoints, uses no example credentials, detects dependency failures, and can restore a known CV and scorecard from backup.

Evidence status: Configuration risk; deployed infrastructure outside this repository was not assessed

```
docker-compose.yml:18–33, 109–116
frontend/nginx.conf:1–31
backend/src/main/java/com/aiinterview/controller/HealthController.java:13–20
ai-service/app/main.py:402–415
```

ACADEMIC COMPLETION

Missing parts for a final-year submission

The repository demonstrates implementation work. It does not, on its own, provide a complete final-year research submission. Check the following against your department's rubric and any documents maintained outside this repository.

Deliverable	What to add or verify
Problem and contribution	A narrow target user/problem, research questions, related-work comparison, and a defensible contribution beyond integrating a language model. A proposed contribution is a traceable CV-to-practice workflow plus controlled recruiter assessments; it needs evaluation.
Requirements / traceability	SRS with functional and nonfunctional requirements; role/use-case matrix; acceptance criteria; and requirement → implementation → test links. Include explicit exclusions.
Design evidence	Current component/deployment diagrams, ERD/cardinalities, sequence diagrams for upload and interview retry/recovery, state machines for application/interview/grading/decision, and API contracts.
Research dataset	An anonymized/synthetic CV set, manually labeled skills, question/answer rubric, independently graded answers, held-out evaluation split and participant information/consent where required.
Evaluation chapter	Extraction accuracy, skill-match limitations, RAG evidence quality, grading agreement/repeatability, usability outcomes, latency/cost, failure cases and limitations. Report measured values only.
Project management	Member contribution matrix, sprint records, risks, decisions and evidence of individual understanding. Use actual work records rather than invented allocation percentages.
Reproducible handover	Pinned environment, sanitized demo accounts/data, fresh-install guide, troubleshooting, backup/restore notes, user guide, test report and demo video.

Research questions your group can realistically answer

• How accurately does the system extract and normalize skills across different CV layouts?

• Does retrieved guide context improve the relevance and evidence support of practice questions compared with the same model without retrieval?

• How closely do AI rubric scores agree with independent human graders, and how much do they vary across repeated runs?

No measured scoring-validity study, labeled benchmark or completed usability study was found in the reviewed repository. That means evidence is missing here, not that such work cannot exist elsewhere.

PROPOSED EXPERIMENT DESIGN — NOT MEASURED RESULTS

A manageable AI evaluation study

Choose a small, carefully labeled evaluation set and keep the scoring rubric fixed. Separate development examples from the final held-out evaluation set.

1. CV extraction and skill matching

A practical pilot is 30–50 synthetic or consented, anonymized CVs across 3–4 target roles. Include single/two-column layouts, scanned pages, fresh graduates, experienced applicants, abbreviations and negative evidence such as "no Java experience." These sample sizes are planning suggestions, not statistical sufficiency claims.

Have two group members independently label explicit skills and important fields; resolve disagreements. Save the exact PDF, extracted text, expected labels and final analysis. Measure skill precision, recall and F1; report OCR success/failure separately from model extraction.

Metric	Meaning in this project
Skill precision	Correctly extracted skills / all extracted skills. Reveals invented or incorrectly normalized skills.
Skill recall	Correctly extracted skills / all labeled explicit skills. Reveals missed skills.
Skill F1	2 × precision × recall / (precision + recall); define zero-denominator behavior.
Matching interpretation	Report matched / required skill count as coverage of listed evidence. It is not a probability of getting hired and does not prove proficiency.

2. Grading against human assessments

Build around 20 approved questions with 3 answer qualities each (roughly 60 responses): correct, partly correct and wrong/off-topic. Include concise correct answers and fluent wrong answers. Ask at least two suitably knowledgeable graders to score independently using the same four-category rubric; compare their agreement before using a reconciled reference.

Report category and overall mean absolute error (AI versus reference), rank agreement, large disagreements and within-human disagreement. Repeat a subset 3–5 times to measure variability. Keep failed evaluations separate from earned zero scores; report their frequency.

Proposed pilot gates: no fabricated score on malformed output; no strong score for clearly irrelevant answers; review cases with errors above 20 points on the 0–100 scale. Numerical accuracy targets should be agreed with your supervisor before running the final evaluation.

AI EVALUATION, CONTINUED

Prove the value of retrieval and responsible scoring

3. Compare retrieval fairly

Run the same held-out prompts under: (A) the configured model without retrieved guides; (B) model plus Chroma retrieval; and optionally (C) model plus recruiter-approved key points for the fixed-question scoring flow. Keep these as separate experimental conditions. Do not claim that fixed key points are the same retrieval method.

For each condition, record actual model, prompt version, embedding model, corpus hash, chunk settings, top_k, retrieved passages, answer, latency, error state and available token/cost metadata. Correct F11 before studying different k values.

Question	Suggested evidence
Was retrieval relevant?	A reviewer labels whether retrieved chunks contain the concept needed. Use a small set of query-to-relevant-chunk labels and report recall@k or hit@k.
Did the output use its sources?	Manually check support for important answer/feedback claims and source consistency. A model-supplied “references_used” flag alone is not proof of support.
Did RAG improve the experience?	Blind reviewers rate relevance, technical correctness and specificity of A/B outputs. Show both improvements and regressions.
Can it say evidence is insufficient?	Include out-of-domain and missing-knowledge queries; check whether the response admits limits rather than presenting weakly related chunks as certainty.

4. Fairness and adversarial cases

Keep qualifications/answers fixed while varying names, CV layout or writing style in a small paired test. Compare score changes and review explanations. Fixed job questions reduce one source of variation but do not establish unbiased grading. Treat browser paste/tab signals as context for human review; they are client-reported, bypassable and do not prove cheating.

Extend the existing answer-fencing tests with CVs and answers that contain instructions, fabricated qualifications or attempts to dictate a score. Test both extraction and grading. Prompt instructions are useful controls, but they require behavioral evaluation; RAG alone does not remove prompt-injection risk. [R2]

The study design above is proposed specifically for this project. NIST’s generative-AI profile is a useful broader risk-management reference [R3]. Lewis et al. provide foundational RAG literature [R4]; this application is not a reproduction of their trained research model.

ACCEPTANCE AND INTEGRATION PLAN

The tests your current suite still needs

Keep the passing unit/component tests. Add a small integration and browser suite around the boundaries where the serious findings occur. Playwright is a suitable option for Chromium/Firefox/WebKit browser testing [R5]; no browser E2E suite was found here.

Scenario	Required pass condition
Two users / all roles	Another account's answer, finish, resume, scorecard, CV, application and recruiter resources are inaccessible; no downstream AI mutation occurs.
Complete real journey	Fresh account → CV → analysis → application → shortlist → fixed interview → result → published decision works with real PostgreSQL and Redis.
Retries and overlap	Lost response, double-click, repeat finish, two tabs, parallel starts and repeated answers do not create duplicate turns or extra attempts.
Server-enforced time	Reload/suspend/direct API calls cannot extend exam deadlines; an accommodation setting is consistently honored.
Provider and database failure	Timeout, rate limit, malformed JSON and save failure produce recoverable states; incomplete grades do not enter averages.
Restart and expiry	AI restart preserves Redis sessions; Redis loss restores supported fixed sessions from durable records. Practice limitation is explained clearly.
Fresh / upgrade database	Flyway runs on an empty PostgreSQL database and supported existing schema. Use disposable instances and sanitized backups; rehearse V3's legacy table replacement.
PDF corpus	Text, scanned, mixed, multilingual/unsupported, password-protected, corrupt and excessive-page fixtures produce documented outcomes without resource stalls.
Browser usability	Keyboard navigation, speech unavailable/denied, mobile viewport, screen-reader labels, focus, contrast and printing are checked.
Modest concurrent load	Measure p50/p95 latency, errors, memory and model calls for a stated workload, e.g. 10 simultaneous sessions. Treat this as a pilot, not a capacity guarantee.

Run real migrations and relational constraints against PostgreSQL, rather than assuming service mocks prove database behavior. Save test environment, commands, results and failing traces as assessment evidence.

OPERATIONAL COMPLETION

A reliable demonstration and deployment

Fresh demonstration preparation

• On a separate disposable setup, follow only the final README. Confirm Java/toolchain alignment, environment placeholders, migrations, OCR native data, embedding availability and initial Chroma indexing.

• Prepare synthetic candidate CVs and separate candidate/recruiter/admin accounts. Demonstrate both practice and fixed job interviews, including one controlled failure and recovery.

• Use Mailpit as a local mail catcher during the demo; describe it as such. Real outbound delivery needs a configured provider and an independently tested deployment.

• Record a short backup demo video and retain approved questions, corpus and a sanitized sample scorecard. These are contingency materials, not replacements for a live system.

Data lifetime and resilience

Data	Current location / recommended policy work
Uploaded PDF and extracted CV	Upload volume and PostgreSQL; browser sessionStorage also holds CV text. Explain retention and deletion behavior, including submitted-CV restrictions.
Live interview context	Redis with idle TTL. Compose disables Redis disk persistence; fixed sessions can be reconstructed from database records, while practice restoration is limited.
Results / decisions	PostgreSQL, including snapshots, transcripts and hiring events. Define retention separately from live-session expiry and test access/export requests.
Provider prompts	Sent to Gemini when models run. Minimize identifiers and describe actual provider handling for the account/tier your group uses; do not make unverified privacy promises.
Knowledge guides	Repository documents plus derived Chroma volume. Keep corpus versions and review provenance/licensing and technical currency.

Back up PostgreSQL and uploaded PDFs as one recoverable set. Test restoring a known application, CV and scorecard. Keep a rebuild procedure for derived vector data. Do not edit previously applied Flyway migration files to fix new problems.

Add dependency-aware readiness, request correlation and basic latency/error/AI-call monitoring. JwtAuthenticationFilter currently catches all exceptions silently (lines 65–67): distinguish invalid tokens from unexpected database faults without logging raw tokens or CV contents.

UX, CLARITY AND MAINTAINABILITY

Useful improvements without expanding the project too far

Suggestion	Why it helps / sensible acceptance check
Explicit processing states	Expose upload/extraction/analysis/review separately and preserve the uploaded resume ID for retry. Avoid Date.now() as a replacement for a missing server ID (CVUpload.jsx:214); a missing ID is a failed contract.
Consistent skill labels	Backend summary bands use 80/60 in skill_gap/analyzer.py; the UI uses 70/45 in SkillGap.jsx. Align or explain these different labels so one percentage does not receive conflicting interpretations.
Avoid stale field results	SkillGap.jsx can start overlapping field analyses without cancellation or a latest-request guard (lines 83–104). A slow old response can overwrite the newest selection. Test rapid role changes with reversed response order.
Clear reset behavior	CVUpload handleReset clears selected analysis/field keys but keeps resumeId/resumeText and review state (lines 287–297). Define “reset” and clear the corresponding state consistently. Verify navigation after resetting.
Accessibility evidence	Keep existing labels, focus behavior and skip link. Test keyboard-only use, speech alternatives, reduced motion, zoom and score meaning without color alone. Use WCAG 2.2 as a testing reference [R6], without claiming certification.
Manage token exposure	JWT is stored in localStorage (api.js:12). This is an XSS exposure consideration, not proof of an existing XSS bug. Review CSP, third-party scripts and session design; a cookie migration would also require CSRF design. [R7]
Maintainable boundaries	InterviewRoom.jsx has 2,481 lines; main.py 2,428; question_generator.py 2,566. After behavior is stabilized, separate UI/session orchestration and AI generation/retrieval/state logic behind clear contracts.
Pagination and query work	Job/application/history endpoints often return complete lists. Add pagination and measure query counts for recruiter views; verify a suspected N+1 pattern before reporting it as a measured performance defect.

Scope decisions

Prioritize reliable analysis, fair assessments, recovery, evaluation and documentation. An avatar, webcam face tracking, a code-execution sandbox, multilingual voice support or a mobile app are optional extensions. Revisit them only if required by the approved scope and there is time to test them properly.

SUGGESTED OWNERSHIP — ADAPT TO YOUR TEAM SIZE

A four-week completion plan for the group

Use workstreams rather than assuming a particular number of members. One person can cover multiple workstreams; each important change should have a second reviewer. This is a proposed schedule, not a statement about your submission deadline.

Week	Backend / AI	Frontend / QA	Research / documentation
1	F01 ownership; design durable command + idempotency flow (F02/F03). Agree grading states and deadline policy.	Write acceptance scenarios; add two-account and retry E2E tests; display pending/recovery states.	Freeze scope and requirements; prepare anonymized dataset and rubric; align architecture.
2	F04–F09: deadlines, grading validation/retry, RAG bootstrap, restricted rebuild and persisted analysis retry.	Connect deadline and processing states; test reload, expiry and failed grading across full journeys.	Run initial CV labels and human grading pilot; review corpus provenance and failure cases.
3	F10–F14: consistent adaptation, retrieval contract, workload limits, environment reproducibility.	Browser/accessibility checks, PDF fixture matrix and modest concurrency test.	Run held-out extraction, RAG A/B and grading agreement/repeatability study; analyze results.
4	Close remaining high-priority defects; verify migrations, backup restore and deployment profile.	Fresh install + complete demo rehearsal; record tests, known limitations and contingency video.	Finalize dissertation, diagrams, user guide, contribution evidence, presentation and viva practice.

Exit criteria for each workstream

• **Backend:** authorized access before AI calls; one accepted answer per turn; recoverable database/provider failures; stable application/interview state transitions.

• **AI:** complete schema validation, explicit failed grades, correct retrieval configuration, tested short/long answer behavior, logged experiment versions and measured quality.

• **Frontend/QA:** a user can recover without duplicating work; time survives reload; key journeys pass in a real browser; error messages and printable results remain usable.

• **Research/documentation:** every major claim has evidence, diagrams match code, results are reproducible and limitations are stated clearly.

If time is shorter, finish the access-control and integrity work plus a smaller honest evaluation. Defer optional features and report the remaining limits explicitly.

DEMONSTRATION AND VIVA

How to present and defend the project

Suggested 10–12 minute demonstration

• **1–2 min:** define the problem, intended users and what the group implemented; show the current architecture and data flow.

• **3 min:** upload a synthetic CV, explain extracted skill evidence and match coverage, then show a brief adaptive practice and learning feedback.

• **3 min:** recruiter-approved fixed questions, candidate application/shortlist, interview result and a manually published hiring decision.

• **2 min:** show evaluation numbers and one failure/recovery case. Explain what the numbers establish and what remains uncertain.

• **1 min:** summarize actual member contributions and the most important limitations.

Likely viva question	Evidence your group should be ready to show
What is your contribution?	A precise user problem, implemented workflow and measured benefit compared with a simpler baseline. API integration alone is not a validated research result.
Why RAG? What is indexed?	The guide corpus, chunking/embedding/retrieval settings, actual retrieved passages, and an A/B comparison. Explain that CV/JD text also enters prompts directly.
Are the scores deterministic?	Skill matching and arithmetic are deterministic for fixed inputs; LLM category scores are not proven deterministic. Show repeat-run variability and human agreement.
How is comparison fair?	Approved fixed job questions, rubric and controlled conditions. Explain remaining model/style/accessibility limitations and human decision-making.
What happens on failure?	Durable answer records, retry/idempotency proof, grading-pending handling, Redis-loss behavior and current recovery limits.
How do you protect CVs?	Owner-scoped APIs, recruiter approval/consent, private service boundary, data minimization, participant information and tested access controls.
How did the group validate it?	Unit/component results plus genuine integration, browser, AI-quality and user-study evidence. Distinguish each category clearly.

Avoid claims such as “100% accurate,” “fully unbiased,” “cheating detection,” or “production-ready” unless the evidence and scope justify them. Prefer a concrete measured result with its dataset and limitations.

USE THIS WITH YOUR SUPERVISOR'S RUBRIC

Final submission checklist

Area	Ready when…
Core product	Candidate preparation and recruiter assessment journeys are demonstrable with stored results; privacy-sensitive examples are synthetic or explicitly consented.
Security / integrity	F01 is closed; F02–F06 have acceptance evidence; candidates cannot mutate another session, replay an answer onto the next question or extend exam time through reload.
AI reliability	Malformed output is unscored; transient scoring failures can recover; the knowledge base initializes cleanly; user queries cannot rebuild it.
Research evidence	Problem statement, related work, contribution, labeled data, baseline, human comparison, repeated runs, charts and limitations are complete.
Tests	Frontend/backend suites remain green; full AI suite runs in the pinned environment; real PostgreSQL/Redis integration and browser flows are recorded.
Operations	Fresh installation works, example secrets are replaced for the intended deployment, backup restoration is demonstrated, and known runtime limits are documented.
Documentation	README/SRS/architecture/ERD/sequences/API/user guide/test report agree on features, Java version, model configuration, data flow and migration state.
Group accountability	Contribution records, peer reviews and demonstration responsibilities are clear. Each member can explain their work and its interaction with the other services.
Presentation	A timed demo, readable evaluation charts, sanitized data, contingency video and honest answers to likely viva questions are prepared.

The most valuable project-level recommendation

Demonstrate that the system handles a real workflow correctly and that its AI feedback is supported by measured evidence. Completing that work will strengthen the project more than adding several unvalidated features.

The review identifies a substantial existing implementation, 16 prioritized findings and several smaller quality opportunities. It does not certify overall completeness because the approved proposal, marking rubric, deployed environment and any external dissertation materials were not supplied.

SOURCES VERIFIED ON 4 NOVEMBER 2026

References and evidence notes

[R1] OWASP API Security: Broken Object Level Authorization

Supports checking object permissions for each API action. The specific F01 finding comes from this repository's control flow, not from the generic guidance.

[R2] OWASP GenAI: Prompt Injection

Supports evaluating injection behavior and treating external/user content as untrusted; retrieved context does not itself eliminate injection risk.

[R3] NIST AI 600-1: Generative AI Profile

Broader voluntary guidance for managing generative-AI risks and evaluation. The pilot dataset sizes and acceptance suggestions in this report are project-specific proposals.

[R4] Lewis et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Foundational related work for retrieval with generation; cite alongside literature specific to CV extraction, interview assessment and LLM grading.

[R5] Playwright official documentation

A possible browser E2E tool supporting Chromium, Firefox and WebKit; no installation or test code was added during this review.

[R6] W3C: Web Content Accessibility Guidelines 2.2

Reference for an accessibility test checklist. Source review and the existing UI tests do not establish conformance.

[R7] OWASP: HTML5 Security Cheat Sheet

Explains client-storage exposure considerations; used for the session-storage design review suggestion, not as evidence of an actual XSS exploit.

Local evidence convention

Finding pages cite repository-relative paths and source line numbers from the reviewed snapshot. Where several line spans are listed, they identify connected stages of a workflow. A future source edit can move those lines. The document does not include .env contents, real tokens, CV records or credentials.

The implementation was treated as the source of truth over stale plans and historical investigation documents. Existing automated tests were run without changing source. The only persistent review deliverable is docs/FINAL_YEAR_PROJECT_REVIEW_2026-11-04.pdf.

F01's authorization recommendation follows the object-permission principle described by OWASP [R1]. The remaining detailed findings are based on repository evidence; experiment sizes, ownership and scheduling are recommendations made for this project.